Probing the Capability Ceiling of an LLM Coding Agent for Volta-Specialized GPU Kernel Optimization: A 30-Problem KernelBench Study on the Tesla V100

Team 37
Parallel Programming Course Project
⟨*National Taiwan University*⟩
Evaluated on the NCHC NVIDIA Tesla V100 cluster
Project category: B — AI-agent analysis and optimization of parallel programs

***Abstract*—We study how far a general-purpose large-language-model (LLM) coding agent can push GPU-kernel performance when it is constrained to a single, deliberately "old" hardware target: the NVIDIA Tesla V100 (Volta, compute capability 7.0), in FP32, with Ampere/Hopper features (TF32, native BF16, `cp.async`, WGMMA) explicitly forbidden. Using a fixed 30-problem subset of KernelBench—15 single operators (Level 1), 10 fused operators (Level 2), and 5 end-to-end models (Level 3)—we drive a coding agent (GitHub Copilot Chat) through a strict four-step chain-of-thought protocol that forces an operator characterization, a memory/tiling plan, and a bank-conflict/divergence analysis *before* any code is emitted. Each generated `ModelNew` is compiled and verified for correctness via `torch.allclose` and timed against both PyTorch eager and `torch.compile` (Inductor) baselines measured on the same V100. All 30/30 solutions pass correctness. The agent reaches a geometric-mean speedup of 0.74× over eager among correct solutions, with 57% (17/30) of problems at or above parity (`fast_1`) and headline wins up to 1.7× (RMSNorm), 1.45× (fused convolution + GroupNorm), and 1.5× (causal attention). Our central finding is a sharp, architecture-induced capability ceiling: on V100 FP32 the agent reliably *wins* on memory-bound operators (softmax, RMSNorm, depthwise convolution, attention) by fusing passes and avoiding library round-trips, but reliably *loses* on any compute-bound path dominated by cuBLAS/cuDNN, where the vendor libraries already sit at 90–95% of FP32 peak. Every problem is solved with a hand-written kernel under the same protocol; the five Level-1 convolution/attention problems dominated by cuDNN or the fused-SDPA path are no exception. They are all numerically correct but lose by 0.08–0.40×, and appear in the same single per-problem table as every other problem. We also document three non-obvious harness pitfalls (a static-checker false negative for Triton, an out-of-memory `allclose`, and a dropout RNG mismatch) and the agent-side fixes that were required for a fair evaluation.**

***Index Terms*—GPU kernels, LLM coding agents, KernelBench, Triton, CUDA, V100, Volta, performance optimization, memory-bound, roofline.**

I. Introduction

Hand-writing high-performance GPU kernels is among the most expertise-intensive tasks in systems programming: it couples numerical algorithm design with a detailed mental model of the memory hierarchy, occupancy, and instruction-level parallelism of a specific architecture. The recent generation of LLM coding agents can already emit syntactically valid CUDA and Triton; the open question is whether they can emit *fast* kernels, and—crucially—*where* their performance ceiling lies relative to highly tuned vendor libraries.

This report (project **category B**: *AI-agent analysis and optimization of parallel programs*) attacks that question under an intentionally adversarial constraint. Rather than evaluating on the newest GPU, we pin the target to the **NVIDIA Tesla V100-SXM2-32GB** (Volta, CC 7.0) and to **FP32**, and we forbid every post-Volta acceleration path: TF32, native BF16 tensor cores, asynchronous copies (`cp.async`), and Hopper WGMMA. This choice is deliberate. On V100 FP32 the vendor libraries (cuBLAS, cuDNN) are extremely mature and already operate near the hardware roofline, so any speedup the agent earns must come from genuine algorithmic or memory-traffic improvements rather than from opportunistically switching to a lower-precision tensor-core path. The constraint therefore turns the benchmark into a clean probe of the agent's *reasoning* about the memory hierarchy.

Contributions.

- A reproducible pipeline that drives an LLM coding agent through a four-step chain-of-thought (CoT) protocol over a *fixed* 30-problem KernelBench subset spanning single operators, operator fusion, and end-to-end models, and evaluates each result on a real V100 against both eager and `torch.compile` baselines.
- A full V100 FP32 measurement: 30/30 correct, geometric-mean speedup 0.74× over eager, `fast_1` $= 57\%(17/30)$, with the complete per-problem table and the official `fast_p` curve.
- An architecture-grounded account of *where the agent wins and loses*: a memory-bound "win zone" versus a cuBLAS/cuDNN-dominated "ceiling zone," supported by

roofline reasoning.
- Documentation of three evaluation-harness pitfalls and the agent-side mitigations required for a fair, leak-free comparison—useful to anyone benchmarking LLM-generated kernels.

II. Background and Related Work

A. KernelBench and the `fast_p` metric

KernelBench [1] is a benchmark for LLM-generated GPU kernels. Each problem ships a PyTorch reference module `Model`; a candidate must provide a drop-in `ModelNew` with an identical input/output interface. The harness checks numerical correctness with `torch.allclose` over several randomized inputs and then measures wall-clock runtime. The recommended aggregate metric is `fast_p`: the fraction of problems that are simultaneously correct *and* achieve a speedup of at least p over the baseline. `fast_1` thus measures "at least parity," while `fast_2` measures "at least 2×." We report `fast_p` against both the PyTorch eager baseline and the `torch.compile` (Inductor) baseline.

B. The Volta (V100) target

The V100 provides 80 SMs, 64 KB of register file and up to 96 KB of shared memory per SM, and roughly 900 GB/s of HBM2 bandwidth. Its tensor cores accelerate FP16 matrix multiply, but it has *no* TF32 or native BF16 path and no asynchronous global-to-shared copy. For FP32 workloads the practical performance envelope is therefore set by (i) the ∼15.7 TFLOP/s FP32 peak for compute-bound kernels and (ii) the ∼900 GB/s HBM2 bandwidth for memory-bound kernels. The arithmetic intensity at which a kernel transitions from memory-bound to compute-bound (the roofline ridge point) is $\approx 15{,}700/900 \approx 17$ FLOP/byte. Most elementwise, normalization, reduction, and attention-epilogue operators sit far to the left of this ridge and are bandwidth-limited; dense GEMM and convolution sit to the right and are compute-limited, which is precisely where cuBLAS/cuDNN are hardest to beat.

C. Triton and `torch.compile`

Triton [2] is a Python-embedded DSL that compiles tile-level programs to GPU code, exposing block sizes, shared-memory staging, and software pipelining while hiding register allocation. `torch.compile` with the Inductor backend automatically generates Triton for many subgraphs and is a strong, fully automatic baseline; beating it (not merely eager) is the harder and more meaningful bar, and we report both.

III. Methodology

A. Hardware and software environment

All measurements use one NVIDIA Tesla V100-SXM2-32GB on the NCHC cluster, driver 535.x, CUDA toolkit 12.8, PyTorch 2.5.1 (cu121), and Triton 3.1. Kernels are restricted to Volta-legal constructs; the GPU architecture flag passed to the evaluator is `["Volta"]` and the working precision is FP32 throughout.

B. Fixed problem set

The 30 problems are fixed in advance (file `tasks.txt`) to remove any cherry-picking degree of freedom: 15 Level-1 single operators (GEMM variants, softmax, RMSNorm, reductions, several convolution types, masked cumsum, scaled dot-product attention), 10 Level-2 fused operators (e.g. Conv+ReLU+Bias, Gemm+activation+reduction chains, Conv+Scale+Sigmoid+GroupNorm), and 5 Level-3 end-to-end models (MLP, Vision Transformer, minGPT causal attention, a minGPT block, and a Mamba-2 SSD layer).

C. The four-step chain-of-thought agent protocol

The agent (GitHub Copilot Chat) is governed by a role/system prompt (`PROMPT.md`, reproduced in Appendix A) that casts it as an HPC specialist and *forbids emitting code before analysis*. For every problem it must output, in order:

1) **Operator characterization** — the math, the dtype-/shape, and a compute-bound vs. memory-bound classification with an arithmetic-intensity argument.
2) **Memory and tiling strategy** — V100 block/grid tiling, shared-memory and register budget, and the coalescing pattern.
3) **Hardware-conflict reduction** — avoidance of shared-memory bank conflicts and branch divergence, and a plan for ILP/occupancy.
4) **Implementation** — the Triton or CUDA `ModelNew`.

On failure (compile error, numerical divergence, or an unfavorable profile) the human relays the harness output verbatim and the agent performs a targeted "blind-spot" diagnosis and revises. The complete prompt/response transcript (37 agent responses across three sessions) is exported as raw data (Appendix B).

D. Evaluation procedure

Each solution is evaluated in an *isolated subprocess* by `run_eval_all.py`: it loads the KernelBench reference, runs the correctness check (5 randomized trials), times the custom kernel (100 trials, CUDA events), and then times the eager and `torch.compile` references (100 trials each) on the same device. Subprocess isolation is essential because several solutions install module-level monkey-patches (Section IV) whose effects must not leak across problems. Speedup is $t_{\text{baseline}}/t_{\text{kernel}}$; the driver emits a KernelBench-compatible V100 baseline-timing file and computes `fast_p` directly.

E. Project layout and reproducibility

Solutions live in `solutions/level{1,2,3}/`, per-problem engineering logs in `progress/level{1,2,3}/`, and all evaluation artifacts (`eval_all_v100.json`, `RESULTS_v100.md`, baseline timing) under `results/`. All compute runs on a Taiwania 2 GPU node via SLURM; the evaluation is submitted as a batch job:

```
sbatch finalProject_260531/run_eval_all.sbatch
```

which, on the allocated V100 node, exports `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` and runs `run_eval_all.py`.

TABLE I
AGGREGATE V100 FP32 RESULTS OVER THE FIXED 30-PROBLEM SET. FAST_P = FRACTION CORRECT *and* SPEEDUP $\geq p$.

Metric	Value			
Compiled	30/30			
Correct	30/30			
Geo-mean speedup (vs. eager)	0.74×			
Geo-mean speedup (vs. compile)	0.71×			
fast_p	p=1.0	p=1.5	p=2.0	p=3.0
vs. eager	17/30	2/30	0/30	0/30
vs. compile	11/30	2/30	1/30	0/30

IV. Evaluation-Harness Pitfalls and Agent-Side Fixes

A fair comparison required diagnosing three harness behaviors that would otherwise corrupt the results. We document them because they are general to LLM-kernel benchmarking. These constitute the principal *changes made to the agent's outputs / interaction with the harness*; they are listed exhaustively in Appendix C.

(P1) Static checker rejects valid Triton. KernelBench's optional static "anti-cheat" checker only recognizes a CUDA C++ `__global__` entry point and flags otherwise-valid Triton kernels as "missing kernel." We disable it (`check_kernel=False`); the `torch.allclose` correctness check is unaffected.

(P2) Source-introspection failure for `@triton.jit`. The default loader `exec()`s the candidate, after which Triton cannot recover the kernel source for JIT compilation (`OSError: could not get source code`). We select the temp-file loader (`backend=triton`), which preserves the on-disk source.

(P3) OOM in `torch.allclose` on large tensors. For the 6.4 GB softmax problem, the C++ `isclose` materializes ∼4 intermediate FP32 tensors, peaking near 38 GB and OOMing the 32 GB V100. The affected solutions install a chunked, streaming `allclose` (64 MB working set) at import time; it is numerically identical to the original predicate.

(P4) Dropout RNG mismatch. For models containing `nn.Dropout`, the harness runs the reference and the candidate sequentially *without re-seeding* the CUDA RNG between forwards, so the two dropout masks diverge and even a verbatim copy fails at the FP32 tolerance. We neutralize dropout to the identity at the class level (it is a no-op at inference anyway), which makes the comparison deterministic.

V. Results

Table I reports the aggregate metrics and the `fast_p` curve; Table II gives the complete per-problem breakdown. All 30/30 solutions are numerically correct.

A. Level 1 — single operators

The Level-1 results cleanly separate the two regimes. The *memory-bound* operators are the agent's strongest territory: a two-pass streaming RMSNorm and an online (running-max/running-sum) streaming softmax both reach ≈87% of the HBM roofline and beat eager by 1.7× and 1.4× respectively, and a direct 3×3 depthwise convolution beats even cuDNN (up to 2.2× under `torch.compile`). In contrast, every *compute-bound* GEMM variant (large-K, tall-skinny, transposed-A, transposed-both) lands at 0.5–0.8×: cuBLAS FP32 already runs at 90–95% of peak on V100, while a portable Triton tile reaches only ∼55–65% of peak, so a slowdown is the expected and honest outcome. The same logic governs the five convolution/attention problems that cuDNN or the fused-SDPA path dominates (square/asymmetric 2D convolution P50/P56, transposed 3D convolution P61, dilated 1D convolution P76, and scaled dot-product attention P97). Like every other problem, each is solved with a hand-written Triton kernel and reported in the same per-problem table (Table II). All five are numerically correct but lose, from 0.40× (ConvTranspose3D) and 0.26× (asymmetric Conv2D) down to 0.10× (flash-attention), 0.09× (dilated Conv1D), and 0.08× (large-stride Conv2D), because a portable tile reaches only a fraction of cuDNN's im2col+GEMM or the fused-attention throughput on V100 FP32.

B. Level 2 — operator fusion

Fusion is where Triton's value is clearest. The standout is Conv+Bias+Scale+Sigmoid+GroupNorm (1.45× eager): the agent collapses five elementwise/reduction passes into two, eliminating three full HBM round-trips over the activation tensor. Two further wins come from *algebraic simplification* the agent discovered: $y \cdot s + y \equiv y \cdot (s + 1)$ and a clamp/LogSumExp/Mish chain reduced to a single post-pass. The remaining GEMM-driven chains (Gemm+activation, Matmul+Sigmoid+Sum, Gemm+GELU+Softmax) are again bounded near 1.0× because cuBLAS dominates their runtime and cannot be bypassed.

C. Level 3 — end-to-end models

The largest end-to-end wins come from causal attention: replacing the manual $QK^\top \rightarrow$ mask $\rightarrow$ softmax $\rightarrow V$ with `F.scaled_dot_product_attention(is_causal=True)` routes to V100's memory-efficient fused-attention path, removes a 1 GB attention matrix and its HBM round-trips, and yields 1.5× (minGPT attention) and 1.4× (minGPT block)—beating `torch.compile` too, which did not select the fused path. The MLP is GEMM-bound (parity), the Vision Transformer is too small to escape measurement noise (B=2), and the Mamba-2 SSD layer is numerically delicate—its $\exp(\mathrm{cumsum}(\cdot))$ spans many orders of magnitude, so reordering the contraction breaks the FP32 tolerance and the agent must keep PyTorch's contraction path, limiting gains to in-place micro-optimizations.

VI. Discussion: the capability ceiling

Across all three levels a single explanatory axis emerges—arithmetic intensity relative to the V100 roofline ridge—and

TABLE II
PER-PROBLEM V100 FP32 RESULTS. SPEEDUPS ARE BASELINE/KERNEL; ≥ 1.0 IN **BOLD**. "TECHNIQUE" IS THE DOMINANT STRATEGY THE AGENT CHOSE.

Lv	PID	Operator	Technique	Correct	Kernel (ms)	Eager (ms)	Sp. eager	Sp. compile
1	6	Matmul (large K)	Triton split-K; cuBLAS-bound	✓	8.850	4.590	0.52×	0.55×
1	9	Tall-skinny matmul	Triton tile; cuBLAS-bound	✓	7.850	6.310	0.80×	0.80×
1	16	Matmul $A^\top B$	Triton tl.trans	✓	13.800	9.660	0.70×	0.70×
1	18	Matmul $A^\top B^\top$	Triton double trans	✓	15.400	9.090	0.59×	0.59×
1	23	Softmax (wide row)	streaming online softmax	✓	24.700	34.700	**1.40**×	**1.35**×
1	36	RMSNorm	2-pass streaming	✓	28.700	48.400	**1.69**×	**1.06**×
1	47	Sum reduction	cub-bound	✓	13.300	11.900	0.89×	**1.02**×
1	50	Conv2D (11x11 s4)	Triton implicit GEMM	✓	103.000	7.960	0.08×	0.08×
1	56	Conv2D (asymmetric)	Triton implicit GEMM	✓	82.200	21.500	0.26×	0.31×
1	61	ConvTranspose3D	Triton gather GEMM	✓	77.300	30.900	0.40×	0.40×
1	76	Conv1D (dilated)	Triton implicit GEMM	✓	511.000	44.000	0.09×	0.09×
1	82	Depthwise Conv2D	Triton direct 3x3 (beats cuDNN)	✓	3.840	5.030	**1.31**×	**2.22**×
1	86	Depthwise-sep Conv2D	Triton dw + cuBLAS pw	✓	9.470	10.900	**1.15**×	**1.74**×
1	93	Masked cumsum	fused mask+scan	✓	32.000	31.900	1.00×	0.51×
1	97	Scaled dot-prod attn	Triton flash-attn (D-tiled)	✓	1140.000	109.000	0.10×	0.10×
2	1	Conv+ReLU+Bias	cuDNN + fused epilogue	✓	22.600	24.300	**1.08**×	**1.09**×
2	12	Gemm+Mul+LeakyReLU	cuBLAS-bound	✓	9.820	9.950	**1.01**×	0.97×
2	21	Conv+Scale+Sig+GN	5 passes -> 2	✓	12.400	18.000	**1.45**×	**1.15**×
2	22	Matmul+Clamp+LSE+Mish	algebraic + 1-pass post	✓	9.880	10.300	**1.04**×	0.97×
2	40	Matmul+Scale+Residual	$ys{+}y \to y(s{+}1)$	✓	39.100	40.500	**1.04**×	0.97×
2	45	Gemm+Sigmoid+LSE	two GEMMs dominate	✓	29.500	30.100	**1.02**×	0.98×
2	56	Matmul+Sigmoid+Sum	cuBLAS-bound	✓	20.000	20.200	**1.01**×	**1.01**×
2	66	Matmul+Dropout+Softmax	dropout-RNG fix	✓	5.000	4.980	1.00×	**1.00**×
2	88	Gemm+GN+Swish+Mul+Sw	5 passes -> 2	✓	10.400	10.700	**1.03**×	0.96×
2	99	Gemm+GELU+Softmax	cuBLAS-bound	✓	9.920	10.000	**1.01**×	0.97×
3	1	MLP	cuBLAS-bound; in-place ReLU	✓	12.800	12.800	**1.00**×	0.98×
3	28	Vision Transformer	small B; SDPA	✓	3.560	2.630	0.74×	0.83×
3	43	minGPT causal attn	SDPA is_causal	✓	28.900	43.500	**1.51**×	**1.21**×
3	44	minGPT block	SDPA + gelu(tanh)	✓	77.200	107.000	**1.39**×	**1.06**×
3	48	Mamba-2 (return Y)	keep einsum path	✓	23.900	25.200	**1.05**×	0.69×

it predicts the agent's success better than problem "difficulty" or level.

The win zone (memory-bound). When a kernel is bandwidth-limited, the optimal strategy is to minimize HBM traffic, and this is exactly the kind of reasoning the four-step CoT elicits well: count the passes, fuse them, keep intermediates in registers/SRAM, stream rows that exceed shared memory. Softmax, RMSNorm, depthwise convolution, fused GroupNorm epilogues, and fused attention all fall here, and the agent earns 1.1–2.2×.

The ceiling zone (compute-bound, library-dominated). When runtime is dominated by dense FP32 GEMM or by a cuDNN convolution, the agent cannot win, because the comparison is no longer "LLM vs. naïve PyTorch" but "LLM-written tile vs. a decade of vendor assembly tuning already at 90–95% of peak." Here a portable hand-written kernel does not merely fail to win—it loses by large margins (0.08–0.40×; the convolution/attention problems in Table II, P50/P56/P61/P76/P97). The metric therefore reflects that an LLM-written tile remains far from the vendor library on these compute-bound paths on V100 FP32.

Algorithmic rewrites beat micro-optimization. The biggest end-to-end gains came not from clever tiling but from higher-level rewrites the agent reasoned about: choosing the fused-attention API, applying the causal mask implicitly, and algebraically folding elementwise chains. On a precision-locked V100 this "choose the better algorithm/dispatch" axis, not raw kernel craftsmanship, is where an LLM agent adds the most value.

VII. THREATS TO VALIDITY

Measurement noise. Small Level-3 models (e.g. the ViT, $B{=}2$) have runtimes near 2–3 ms with $\sim$10% standard deviation, so sub-10% deltas are within noise; we report eager and compile baselines to bracket this. **Baseline choice.** `fast_p` depends on the baseline; we publish both eager and `torch.compile` curves rather than the more favorable eager-only number. **Compute-bound problems.** The five Level-1 convolution/attention problems (P50/P56/P61/P76/P97) are handled with the same hand-written-kernel protocol as every other problem; all five are correct but lose outright (0.08–0.40×) because cuDNN/SDPA dominates these compute-bound paths on V100 FP32. **Agent identity.** Results characterize one agent (GitHub Copilot Chat) under one prompt protocol; absolute numbers may shift with other models, though the roofline-driven win/ceiling structure should persist.

VIII. CONCLUSION

Under a precision- and architecture-locked V100 regime, an LLM coding agent driven by an analysis-first protocol

produced 30/30 numerically correct solutions and a geometric-mean speedup of 0.74× over eager, with 57% (17/30) of problems at or above parity. The results expose a clean capability ceiling: the agent is a reliable optimizer for memory-bound operators—where fusing passes and cutting HBM traffic yields up to 2.2×—but it cannot, and should not be expected to, beat mature cuBLAS/cuDNN FP32 paths on compute-bound problems. The most transferable lesson is that on legacy hardware the highest-leverage move for an LLM agent is algorithmic: recognizing the roofline regime and either fusing/streaming (memory-bound) or dispatching to the optimal library (compute-bound), rather than attempting to out-tile a vendor library.

APPENDIX A
ROLE / SYSTEM PROMPT

The verbatim role/system prompt that governs the agent is provided as raw data in `finalProject_260531/PROMPT.md`, with the fixed problem list in `tasks.txt` and the pipeline description in `PIPELINE.md`. It defines the HPC-specialist persona, the V100 constraints, the fixed 30-problem set, the four-step CoT requirement, and the output-path conventions.

APPENDIX B
RAW DATA MANIFEST

All raw data accompanies this report under `finalProject_260531/`:

- **Input prompts & outputs:** the complete agent transcript (input prompts + agent responses) is exported to `report/raw_data/agent_conversation_log.md` (3 sessions, 37 responses) by `report/raw_data/export_conversation.py`.
- **System/role prompt:** `PROMPT.md` (Appendix A).
- **Agent outputs (kernels):** `solutions/level{1,2,3}/*.py` (30 `ModelNew` implementations) plus per-problem logs in `progress/`.
- **Measurements:** `results/eval_all_v100.json`, `results/RESULTS_v100.md`, and the V100 baseline timing under `results/timing/V100_SXM2_32GB_NCHC/`.

APPENDIX C
CHANGES MADE TO THE AGENT / HARNESS

The full list of agent-side and harness-interaction changes (the four pitfall fixes P1–P4, the evaluation flags, the prompt edits, and the persisted "eval-tricks" notes) is documented in `report/raw_data/agent_changes.md`.

REFERENCES

[1] A. Ouyang *et al.*, "KernelBench: Can LLMs Write Efficient GPU Kernels?," *arXiv:2502.10517*, 2025.

[2] P. Tillet, H. T. Kung, and D. Cox, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," in *MAPL*, 2019.

[3] T. Dao *et al.*, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *NeurIPS*, 2022.

[4] NVIDIA, "NVIDIA Tesla V100 GPU Architecture," Whitepaper WP-08608, 2017.